A TRAINING REPORT

on

CLAIMS AUTO-ADJUDICATION ENGINE FOR HEALTH INSURANCE

A Hybrid Rule-Engine and LLM System for the ReAssure 3.0 Product

Submitted in partial fulfilment of the requirements for the award of the degree of

[DEGREE AND BRANCH]

Submitted by

[YOUR NAME]

Roll No.: **[ROLL NUMBER]**

Training carried out at

[ORGANISATION NAME]

[INTERNSHIP START DATE] to **[INTERNSHIP END DATE]**

Under the guidance of

[INDUSTRY MENTOR NAME], **[MENTOR DESIGNATION]**

[FACULTY GUIDE NAME], **[FACULTY DESIGNATION]**

[DEPARTMENT NAME]

[COLLEGE NAME], [CITY]

[ACADEMIC YEAR]

Certificate

This is to certify that the training report titled **"Claims Auto-Adjudication Engine for Health Insurance"** submitted by **[YOUR NAME]** (Roll No. **[ROLL NUMBER]**) in partial fulfilment of the requirements for the award of the degree of **[DEGREE AND BRANCH]** at **[COLLEGE NAME]** is a record of the work carried out by the student during industrial training at **[ORGANISATION NAME]** from **[INTERNSHIP START DATE]** to **[INTERNSHIP END DATE]** under my supervision.

The matter embodied in this report has not been submitted for the award of any other degree or diploma.

[FACULTY GUIDE NAME] **[HEAD OF DEPARTMENT NAME]**
Faculty Guide Head of Department

Declaration

I, **[YOUR NAME]**, hereby declare that the training report titled **"Claims Auto-Adjudication Engine for Health Insurance"** is an authentic record of the work I carried out during my industrial training at **[ORGANISATION NAME]** from **[INTERNSHIP START DATE]** to **[INTERNSHIP END DATE]**. The design, implementation and testing described here are my own work, done under the guidance of **[INDUSTRY MENTOR NAME]** and **[FACULTY GUIDE NAME]**. Sources of information used are acknowledged in the references.

Product specifications, policy wordings and rule documents referred to in this report belong to the organisation and its client, and are reproduced only to the extent needed to explain the work.

Place: [CITY]
Date: [DATE]

[YOUR NAME]
Roll No. [ROLL NUMBER]

Acknowledgement

I am grateful to **[ORGANISATION NAME]** for giving me the opportunity to work on a real production problem during my training. I thank my industry mentor, **[INDUSTRY MENTOR NAME]**, for explaining the claims domain patiently, for reviewing my design decisions, and for pushing me to make every automated decision explainable.

I thank my faculty guide, **[FACULTY GUIDE NAME]**, and the Head of Department, **[HEAD OF DEPARTMENT NAME]**, for their support and for the flexibility that made this training possible. I also thank the members of the policy automation team who answered my questions about ReAssure 3.0 policy wordings, claim workflows and the company's systems.

Finally, I thank my family and friends for their encouragement throughout the training period.

[YOUR NAME]

Abstract

Health insurance claims in India are still largely adjudicated by people who read a claim, read the policy wording and work out what is payable. For a feature-rich product such as ReAssure 3.0, with three variants, optional riders, waiting periods, a room-rent pro-rata clause, stacked co-payments and lifetime benefits such as ReAssure Forever and Lock the Clock, this is slow and inconsistent between processors.

This report describes the design and implementation of a claims auto-adjudication engine built during my training. The engine encodes **92 product rules** extracted from the policy wording into a versioned rule store. For each claim line item an AI planner selects the applicable rules, orders them as a directed acyclic graph using Kahn's algorithm, and runs them through a **seven-gate pipeline**: policy, member, coverage, waiting periods, exclusions, financial computation and state update. All money is computed by deterministic, property-tested calculators. A large language model is used only where language must be understood, for example to decide from a discharge summary whether a treatment is cosmetic, maternity-related or diagnostic-only. Every LLM verdict carries a confidence score; a four-tier routing policy sends low-confidence or incomplete cases to human queues instead of deciding them automatically.

The system is exposed through an asynchronous FastAPI service with Server-Sent Events streaming, and a React/TypeScript console that shows each rule result live, the financial waterfall and an AI-written explanation. It supports a cloud LLM (Groq, Qwen3.8 27B), a locally hosted model through llama.cpp, and an offline deterministic mode. The codebase has about 14,000 lines of Python and 3,000 lines of TypeScript, and 77 automated tests including property-based tests of the financial invariants. Ten reference scenarios covering approval, partial approval, rejection and assisted review were run end to end against the live LLM. During final testing I found and fixed defects including a pro-rata deduction that could exceed the claimed amount and a review request that was being lost; I also cut LLM calls per line item from about eleven to one by batching exclusion checks.

Keywords: health insurance, claims adjudication, rule engine, large language models, DAG scheduling, explainable AI, FastAPI, React.

Table of Contents

Certificate i
Declaration ii
Acknowledgement iii
Abstract iv
Table of Contents v
List of Abbreviations vii
Chapter 1: Introduction 1
1.1 Background 1
1.2 About the Organisation 1
1.3 Problem Statement 1
1.4 Objectives 1
1.5 Scope 2
1.6 Organisation of the Report 2
Chapter 2: Domain Study and Literature Review 3
2.1 The Health Claim Lifecycle 3
2.2 Policy Concepts the Engine Implements 3
2.3 ReAssure 3.0 Product Features 3
2.4 Approaches to Claim Automation 4
2.5 Explainability and Audit 4
Chapter 3: Tools and Technologies 6
3.1 Python, Pydantic and Typed Data 6
3.2 FastAPI and Asynchronous Programming 6
3.3 Large Language Models 7
3.4 React, TypeScript and UI Design 7
3.5 Testing Tools 7
Chapter 4: Execution Methodology 8
4.1 Approach 8
4.2 Requirement Analysis 8
4.3 Phase Plan and Timeline 9
4.4 Development Workflow for Each Rule 9
4.5 Design Principles 10
4.6 Quality Assurance 10
4.7 Version Control and Documentation 10
Chapter 5: System Design 11
5.1 Architecture 11

5.2 Data Model 11
5.3 Product Memory and Rule Blueprints 12
5.4 AI Planner and DAG Scheduling 12
5.5 The Seven-Gate Pipeline 13
5.6 Semantic Agent 13
5.7 Confidence Routing 14
5.8 API Design 14
5.9 Database and Security 14
Chapter 6: Implementation 15
6.1 Code Organisation 15
6.2 Financial Calculators 15
6.3 Planner 16
6.4 Layer-Parallel Execution 16
6.5 Semantic Agent and Structured Output 16
6.6 Batched Exclusion Screening 16
6.7 Streaming, Documents and Observability 17
6.8 Offline Mode 17
Chapter 7: Testing and Results 18
7.1 Test Strategy 18
7.2 Property-Based Tests of the Money 18
7.3 Scenario Results on the Live LLM 19
7.4 Screenshots 20
7.5 Defects Found and Fixed in Final Testing 22
7.6 Performance 23
Chapter 8: Challenges and Learnings 24
8.1 Challenges 24
8.2 Learnings 24
Chapter 9: Conclusion and Future Scope 25
9.1 Conclusion 25
9.2 Future Scope 25
References 26
Appendix A: Running the Project 27

(If page numbers are not shown, right-click the table and choose **Update Field**.)

List of Abbreviations

Abbreviations used in this report

Abbreviation	Expansion
API	Application Programming Interface
DAG	Directed Acyclic Graph
HDC	Hospital Daily Cash
ICD	International Classification of Diseases
IRDAI	Insurance Regulatory and Development Authority of India
JSON	JavaScript Object Notation
LLM	Large Language Model
OCR	Optical Character Recognition
OPD	Out-Patient Department
PA	Personal Accident
PAS	Policy Administration System
PED	Pre-Existing Disease
SI	Sum Insured
SSE	Server-Sent Events
TPM	Tokens Per Minute
TSD	Technical Solution Design
UIN	Unique Identification Number (of an insurance product)

Chapter 1: Introduction

1.1 Background

A health insurance claim passes through several hands before money is paid. The hospital or the policyholder submits bills and a discharge summary; a claims processor checks that the policy is active, that the member is covered, that the treatment is not excluded, that waiting periods have been served, and then works out how much of the bill is payable after room-rent limits, co-payments, deductibles and sub-limits. Much of this is mechanical, but it depends on reading long policy documents correctly every time.

ReAssure 3.0 (UIN NBHHLIP26047V012526) is a retail health product whose wording runs to over 2,600 lines. It has three variants (Classic, Select and Elite), optional covers such as Borderless, HeadsUp and Tiered Network, and lifetime features such as Booster+, ReAssure Forever, Lock the Clock, Live Healthy and Cash-Bag+. Each feature changes how a claim should be decided. A processor has to keep all of this in mind, which leads to slow decisions and to two processors deciding the same claim differently.

1.2 About the Organisation

[ORGANISATION NAME] is [ONE OR TWO SENTENCES ABOUT THE ORGANISATION: WHAT IT DOES, ITS CLIENTS AND YOUR TEAM]. During my training I worked with the policy automation team on the Claims 2.0 programme for the ReAssure 3.0 product, whose goal is AI-first auto-adjudication of health claims.

1.3 Problem Statement

To design and build a system that takes a claim line item, already extracted from bills and medical documents, together with the policy, member and benefit-balance data, and returns a decision of approve, partially approve, reject or refer to a human. The decision must follow the ReAssure 3.0 rules exactly, compute the payable amount to the rupee, explain every step, and never auto-decide when it is unsure.

1.4 Objectives

- Convert the policy wording and technical solution design into a machine-readable, versioned rule store.
- Build a deterministic engine for every rule that can be expressed as arithmetic or a date comparison.

- Use a large language model only for rules that need clinical or linguistic judgement, with structured and validated output.
- Route decisions by confidence so uncertain or incomplete claims reach the right human queue.
- Produce a complete audit trail: every rule evaluated, every tool call, every LLM prompt and reply.
- Provide an API and a web console that a claims team can use and demonstrate.
- Test the engine thoroughly, including property-based tests of the financial calculations.

1.5 Scope

The engine covers the product, policy and member validation layer of adjudication, as defined in the technical solution design. It receives claim line items that upstream systems have already extracted and medically coded, and produces a structured decision for each line item. The Policy Administration System (PAS) remains the final authority that revalidates and pays. Fraud analytics, tariff negotiation with hospitals and the upstream extraction pipeline are outside the scope, although the project includes a document-upload feature that uses OCR and the LLM to pre-fill claim fields for demonstration.

1.6 Organisation of the Report

Chapter 2 introduces the health-insurance concepts the engine implements and the approaches to claim automation. Chapter 3 lists the tools and technologies I learnt and used. Chapter 4 describes the execution methodology: how the work was planned, carried out and checked. Chapter 5 presents the system design and Chapter 6 the implementation. Chapter 7 covers testing and results, Chapter 8 the challenges and what I learnt, and Chapter 9 concludes with future work.

Chapter 2: Domain Study and Literature Review

2.1 The Health Claim Lifecycle

A claim moves through intimation, document collection, adjudication and settlement. In a cashless claim the hospital requests pre-authorisation and the insurer approves an amount before discharge; in a reimbursement claim the policyholder pays and submits bills afterwards. Adjudication, the subject of this project, is the step where the insurer decides admissibility and the payable amount. The IRDAI master circular on health insurance (2024) sets tight timelines for cashless decisions, which makes manual adjudication a bottleneck and is a strong reason for automation.

2.2 Policy Concepts the Engine Implements

Table 2.1: *Key policy concepts and how the engine handles them*

Concept	Meaning	Where it is handled
Sum Insured (SI)	Maximum amount payable in a policy year	Gate 6 SI waterfall: Base SI, then Booster+, then ReAssure Forever pool
Waiting period	Time from inception before a condition is covered: 30 days initial, 24 months for specified diseases, 36 months for PEDs	Gate 4, Tool 1; porting credit and accident waiver applied
Pre-existing disease (PED)	A condition diagnosed before the policy began	Gate 4 PED rule, using declared PEDs
Exclusion	Treatment never covered, e.g. cosmetic surgery, maternity, rest cure	Gate 5: keyword pre-check, then batched LLM screen
Room-rent pro-rata	If a higher room category is taken, associated expenses are paid in proportion eligible rent / actual rent (clause 6.2.4(d))	Gate 6, Tool 2
Co-payment	Share of the admissible amount borne by the insured; can stack (base, room category, network, late intimation)	Gate 6, Tool 3
Deductible	Annual amount the insured bears before the insurer pays	Gate 6, Tool 4
Pre/post hospitalisation	Expenses 60 days before and 180 days after a covered stay	Gate 6, window validator

2.3 ReAssure 3.0 Product Features

Beyond the standard covers, ReAssure 3.0 has features that carry state across claims and renewals, which is what makes it hard to adjudicate by hand:

- **Booster+** carries forward unused sum insured as a bonus pool, which the SI waterfall draws on after the base SI.

- **ReAssure Forever** refills the sum insured once it has been exhausted by a claim. The engine models it as a state machine (not triggered, triggered, active) persisted between claims.
- **Lock the Clock** keeps the premium age at the entry age until the first claim is paid, so the engine records when the clock unlocks.
- **Live Healthy** and **Cash-Bag+** turn wellness points into wallet credit that can pay co-payments; the engine offsets co-pay from this wallet.
- **Optional covers** such as Borderless (treatment abroad), HeadsUp (planned-hospitalisation intimation) and Tiered Network (a narrower hospital network for a lower premium) add penalties when their conditions are not met.

2.4 Approaches to Claim Automation

Rule engines encode the policy as explicit if-then rules. They are exact and auditable, but they cannot read free text: a rule that excludes cosmetic surgery cannot tell whether a rhinoplasty was cosmetic or reconstructive after an accident without understanding the discharge summary.

Machine-learning classifiers trained on past claims can predict an outcome, but they need large labelled datasets, are hard to explain to an auditor or a regulator, and silently drift when the product changes.

Large language models can read clinical text and apply a written rule to it, but they can produce confident wrong answers and cannot be trusted with arithmetic that must be exact to the rupee.

This project takes a **hybrid** approach. Everything that can be computed is computed deterministically, and the LLM is called only for language understanding, with a strict output schema, a confidence score and a human fallback. The design follows the human-in-the-loop principle common in regulated domains: automation handles the clear cases and people handle the rest.

2.5 Explainability and Audit

An insurer must be able to tell a customer, an ombudsman or the regulator why a claim was cut or rejected. The engine therefore records, for every claim, the ordered list of rules evaluated with inputs, outcome and policy section; every calculator call with its inputs and outputs; and for every LLM call the full prompt, the raw reply, the parsed result, the

confidence and the tokens used. It also exports an interactive graph of the decision for replay.

Chapter 3: Tools and Technologies

The training required learning a broad stack, from asynchronous Python services and LLM integration to a typed React front end. Table 3.1 summarises what was used and why.

Table 3.1: *Technology stack*

Area	Technology	Used for
Language	Python 3.12	Engine, API, calculators, tests
Validation	Pydantic v2, pydantic-settings	Typed claim/decision schemas, .env configuration
Web API	FastAPI, Uvicorn, asyncio	Async REST endpoints, SSE streaming, lifespan start-up
LLM (cloud)	Groq API, Qwen3.8 27B	Exclusion screening, decision summaries, document field extraction
LLM (local)	llama.cpp, Gemma 4 E4B (quantised)	On-premise inference option with GBNF grammar-constrained JSON
Documents	pypdf, Tesseract OCR, Pillow	Text from PDFs and scanned bills
Database	PostgreSQL 16, SQLAlchemy 2 (async), asyncpg, Alembic	Policy, member and balance tables; migrations
Containers	Docker Compose	PostgreSQL and pgAdmin for development
Front end	React 19, TypeScript, Vite, Tailwind CSS	Adjudication console
Testing	pytest, Hypothesis, FastAPI TestClient, Playwright	Unit, property-based, API and UI checks
Tooling	Git, GitHub, VS Code	Version control, branches, review

3.1 Python, Pydantic and Typed Data

The whole engine is driven by two Pydantic models: `ClaimContext`, which holds everything known about a claim, and `ClaimDecision`, which holds the outcome and the audit trail. Pydantic validates every request before any business logic runs, and the same models generate the API documentation automatically. I learnt to model money carefully (rounding at defined points, `NUMERIC(18,2)` in the database) and to keep calculators as pure functions that are easy to test.

3.2 FastAPI and Asynchronous Programming

FastAPI with asyncio lets one server handle many claims at once. LLM calls are slow network operations, so they run in a thread pool (`asyncio.to_thread`) and never block the event loop. Independent rules at the same depth of the DAG run concurrently with `asyncio.gather`. Results are streamed to the browser with Server-Sent Events, so the user sees each rule pass or fail as it happens.

3.3 Large Language Models

I learnt how to make an LLM part of a reliable system rather than a chat tool: write prompts that state the policy rule, its exceptions and the claim facts separately; force JSON output with a schema; validate the reply and treat any invalid reply as a reason for human review; use temperature 0 and a fixed seed for repeatable answers; and log everything. I worked with two serving options, a local quantised Gemma model through llama.cpp and Groq's hosted API, and learnt to design around free-tier rate limits (Section 6.6).

3.4 React, TypeScript and UI Design

The console is a single-page React application written in TypeScript. Typed API contracts mirror the backend schemas, a custom hook manages the streaming connection, and a small design system (colour tokens, Rubik and Hanken Grotesk typefaces, 8-pixel spacing) keeps the interface consistent.

3.5 Testing Tools

pytest runs the suite; a shared fixture replaces the LLM with a deterministic responder so tests run offline and repeatably. Hypothesis generates thousands of random inputs to check that financial invariants always hold. Playwright drives a real browser to run the ten demonstration scenarios and capture the screenshots in this report.

Chapter 4: Execution Methodology

This chapter explains how the work was planned and carried out: where the requirements came from, how they were turned into code, in what order the system was built, and how quality was checked at each step.

4.1 Approach

The project followed an **iterative, specification-driven** approach. The technical solution design (TSD) and the policy wording were the specification. Instead of building the whole system at once, I built it in phases, each ending with a working, tested system that the next phase extended. Phase 1 had no AI at all: a purely deterministic engine whose numbers could be checked by hand. AI was added in Phase 2 only after the deterministic core was trusted. This order made sure the LLM never became a crutch for logic that should be exact.

4.2 Requirement Analysis

Requirements came from four sources, studied before any code was written:

Table 4.1: *Requirement sources*

Source	Content	How it was used
Policy wording	ReAssure 3.0 terms, 2,600+ lines: covers, exclusions, waiting periods, annexures	Ground truth for every rule and formula
Technical solution design	Gate sequence, tool contracts, state model, confidence routing, error handling	System architecture and execution order
Rule extraction JSON	92 rule blueprints, rate tables, mutual-exclusivity constraints	Loaded directly as the product memory
API specification JSON	Shapes of the 8 upstream gateway APIs	Context builder and mock gateways

Each rule in the wording was turned into a **rule blueprint** with an identifier (e.g. `R3_EXCL_007`), the gate it belongs to, its execution type (deterministic, semantic or hybrid), the variants it applies to, its dependencies on other rules, the calculator it needs, and for semantic rules a prompt template. Writing rules as data rather than code means a product change can be shipped as a new versioned JSON file.

4.3 Phase Plan and Timeline

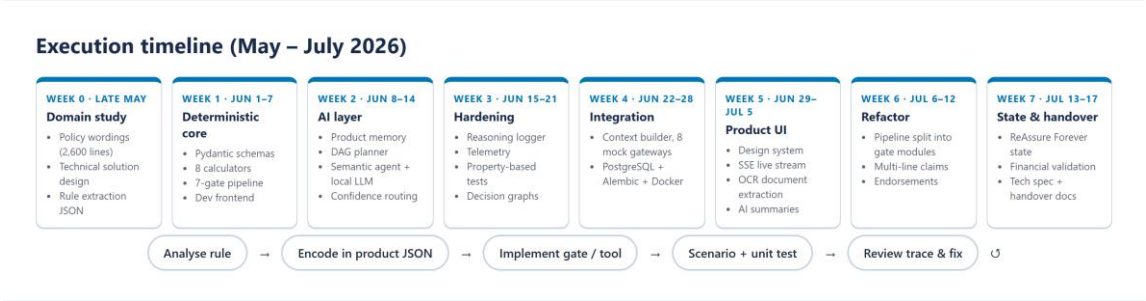


Figure 4.1: *Execution timeline and the per-rule development loop*

Table 4.2: *Phase-wise plan and deliverables*

Phase (weeks)	Work done	Deliverable / exit check
0. Domain study (late May)	Read policy wording and TSD; mapped clauses to gates	Rule inventory; questions resolved with mentor
1. Deterministic core (week 1)	Pydantic schemas, 8 calculators, 7-gate sequential pipeline, developer front end	Hand-checked numbers for sample claims; calculator unit tests
2. AI layer (week 2)	Product memory, DAG planner, semantic agent on local LLM, confidence routing	Compliance matrix against the TSD
3. Hardening (week 3)	Reasoning logger, telemetry, property-based tests, decision-graph export	Invariant tests passing; full audit trail
4. Integration (week 4)	Context builder, 8 mock gateways, PostgreSQL schema, Alembic, Docker	Claim assembled from APIs end to end
5. Product UI (week 5)	Design system, live SSE stream, OCR document extraction, AI summaries	Demonstrable console
6. Refactor (week 6)	Pipeline split into gate modules; multi-line claims; endorsements	Same results after refactor (regression suite)
7. State and handover (week 7)	ReAssure Forever state, financial validation, specification and handover documents	Handover document
8. Final hardening	Groq integration, batched LLM screening, defect fixes from scenario testing, report	77 tests passing; 10 scenarios run on live LLM

4.4 Development Workflow for Each Rule

Every rule went through the same loop, shown at the bottom of Figure 4.1:

1. **Analyse** the clause in the policy wording, including its exceptions (for example, cosmetic surgery is excluded *unless* it is reconstruction after an accident, burns or cancer).
2. **Encode** it as a rule blueprint: gate, execution type, dependencies, variants.
3. **Implement** the gate check or calculator, keeping calculators pure.
4. **Test** it with a unit test and, for money, a property-based test; add or update a scenario preset.

5. **Review the trace**: run the scenario and read the decision trace and reasoning log line by line. Many defects were found here rather than by assertions.

4.5 Design Principles

Table 4.3: *Principles followed throughout the work*

Principle	What it meant in practice
Deterministic first	Money, dates and eligibility are never computed by the LLM
LLM only for language	The LLM judges clinical text against a written rule and returns a verdict, reasoning and confidence
Fail safe to a human	Invalid LLM output, errors, missing data or low confidence route to a review queue, never to an automated decision
Everything traceable	Each step records inputs, outcome, policy section, confidence and timing
Configuration, not code	Rules live in versioned JSON; thresholds and the LLM provider live in .env
Fail fast	A gate that fails stops the pipeline immediately with the reason

4.6 Quality Assurance

- **Automated tests**: 77 tests across 8 files, run after every change (Chapter 7).
- **Compliance matrix**: a component-by-component check of the implementation against each section of the TSD, which identified gaps such as PAS reconciliation being a stub.
- **Scenario presets**: ten realistic claims, each exercising a different rule path, run through the real UI and API.
- **Observability**: per-gate latency, token usage and decisions written to a JSONL telemetry file, used to find slow or failing steps.
- **Mentor review**: design decisions and open questions about the wording were reviewed with the mentor.

4.7 Version Control and Documentation

Work was tracked in Git with separate branches for stable work and experiments (`main`, `change`, `further`). Documentation was written alongside the code: a README with run instructions, an architecture document, a design-system document, a compliance report, and a technical specification and handover document at the end of the training.

Chapter 5: System Design

5.1 Architecture

The system has five layers (Figure 5.1). The presentation layer is the React console. The API layer validates requests and streams results. The adjudication engine plans and executes rules. The intelligence layer contains the semantic agent and the calculators. The data layer holds persistent state and the audit and telemetry records.

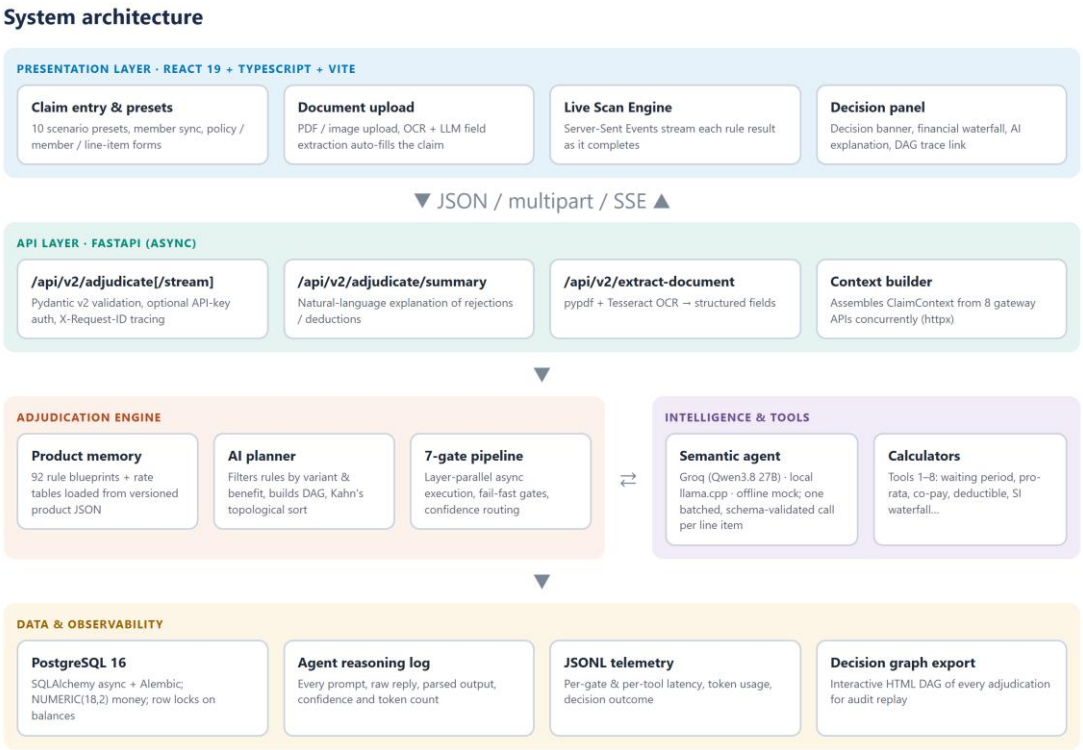


Figure 5.1: *Layered system architecture*

5.2 Data Model

`ClaimContext` is assembled from eight upstream sources and is the single input to the engine. `ClaimDecision` is the single output.

Table 5.1: *Main parts of ClaimContext*

Part	Contents	Source API
policy	Variant, status, dates, sum insured, riders opted, room category, co-pay, deductible	Policy API
member	Age, entry age, relationship, date of addition, declared PEDs	Member API
history	Prior claims, utilised SI, claim-free years	Claims History API
porting	Prior coverage months, waiting-period credit	Porting API
network	Hospital network type, tiered-network membership	Network API

Part	Contents	Source API
benefit_balance	Base SI, Booster+, Forever pool, wallet, deductible used	Benefit Balance API
lifetime_state	ReAssure Forever state, Lock the Clock, once-per-lifetime flags	Lifetime State API
line_items	Description, bucket, amounts, room details, dates, diagnosis, discharge summary	Upstream extraction

`ClaimDecision` contains the overall decision, the claimed, admissible and payable totals, a deduction breakdown (pro-rata, co-pay, deductible, non-payables, penalties), the SI waterfall, per-line-item decisions, the full decision trace, the confidence score, whether manual review is needed and why, and a payload for the PAS.

5.3 Product Memory and Rule Blueprints

Table 5.2: *Rule blueprint fields*

Field	Purpose
rule_id, rule_name	Identity, e.g. R3_EXCL_007 Cosmetic or plastic surgery
gate	Which of the seven gates evaluates the rule
execution_type	DETERMINISTIC, SEMANTIC or HYBRID
variant_applicability	Classic / Select / Elite
depends_on	Rules that must run first; forms the DAG edges
tool_required	Calculator used, if any
semantic_prompt_template	Prompt with placeholders for claim facts
priority, confidence_weight	Ordering within a gate; weight in the overall confidence

5.4 AI Planner and DAG Scheduling

For each line item the planner keeps only the rules that apply to the policy variant and benefit bucket, drops optional covers that were not bought, and builds a graph whose edges are the `depends_on` links. Kahn's algorithm produces a topological order; a priority queue keyed by (gate number, priority, rule id) makes the order deterministic, so the same claim always produces the same plan. The executor then groups steps by their depth in the DAG and runs each depth layer concurrently.

5.5 The Seven-Gate Pipeline

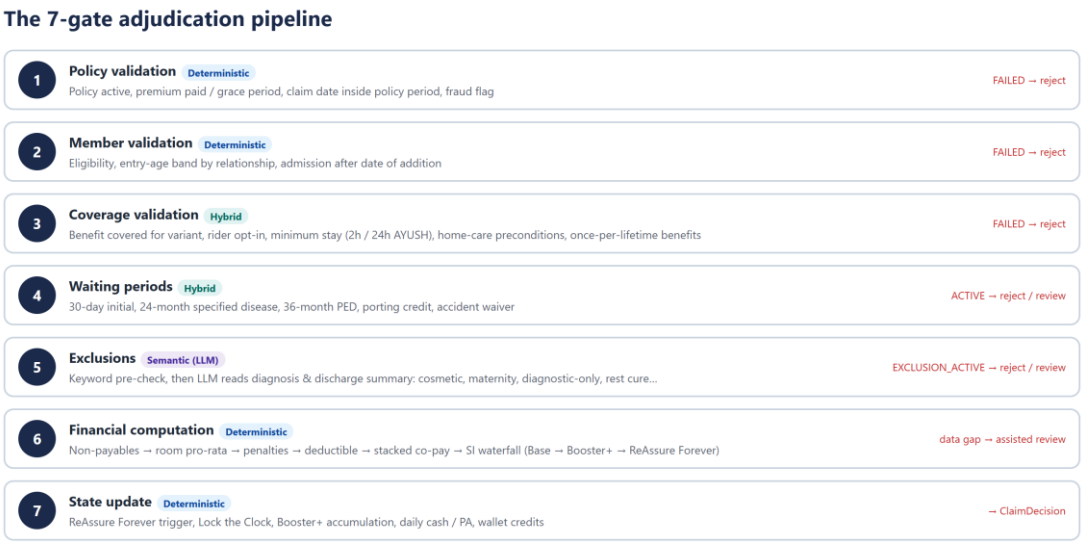


Figure 5.2: *The seven gates and their early-exit behaviour*

Gates 1 and 2 are structural checks that run before the plan; if the policy has lapsed, nothing else is evaluated. Gates 3 to 5 decide admissibility, and Gate 6 computes the money in a fixed order defined by the TSD: non-payable items, room pro-rata, penalties, deductible, stacked co-payment and finally the SI waterfall. Gate 7 updates lifetime state for the next claim.

5.6 Semantic Agent

The semantic agent hides the LLM provider behind one interface. Three providers are available and chosen in `.env`:

- **groq**: hosted inference on Groq (default model Qwen3.8 27B), JSON mode, temperature 0, seed 42.
- **local**: a llama.cpp server running a quantised Gemma 4 model, with a GBNF grammar that forces valid JSON when reasoning output is off.
- **mock**: deterministic keyword rules that read only the claim details; used by the test suite and for offline demonstrations.

Whatever the provider, the reply must validate against a Pydantic schema containing an evaluation status, a reasoning trace and a confidence between 0 and 1. A reply that fails validation becomes a failed step with zero confidence, which routes the claim to review.

5.7 Confidence Routing



Figure 5.3: *Four-tier confidence routing*

Each step contributes a confidence (deterministic steps contribute 1.0), and the line item's confidence is their weighted mean. In addition, Gate 6 can explicitly request review when data needed for a calculation is missing, for example the hospitalisation dates needed to check a post-hospitalisation window.

5.8 API Design

Table 5.3: *Main API endpoints*

Method and path	Purpose
POST /api/v2/adjudicate	Adjudicate a ClaimContext; returns ClaimDecision
POST /api/v2/adjudicate/stream	Same, streaming each DecisionTrace as an SSE event, then the decision
POST /api/v2/adjudicate/summary	Plain-language explanation of a decision, written by the LLM
POST /api/v2/extract-document	Upload a PDF or image; returns extracted claim fields
GET /health, GET /readiness	Liveness and readiness probes
/api/external/... (development)	Eight simulated upstream gateway APIs

5.9 Database and Security

PostgreSQL tables mirror the context parts (policies, members, claims history, porting records, network providers, benefit balances, lifetime states, endorsements). Money columns use `NUMERIC(18,2)`, and balance rows are locked on update to prevent two concurrent claims overdrawing the same sum insured. API keys are compared in constant time, CORS is restricted to the console's origin, internal errors are logged but never returned to the client, and secrets are read only from the environment.

Chapter 6: Implementation

6.1 Code Organisation

Table 6.1: *Main modules*

Module	Responsibility
src/schemas.py	Pydantic models for input, output and per-claim state
src/calculators.py	Tools 1–8: pure calculation functions
src/product_memory.py	Loads rule blueprints and tables; builds prompt templates
src/planner.py	Rule filtering, DAG construction, topological sort
src/pipeline.py	Pipeline entry points (sync, async, streaming)
src/pipeline_modules/	Gate implementations, execution, financials, state, decision composition, graph export
src/semantic_agent.py	LLM providers, prompts, validation, caching, batching
src/mock_llm.py	Deterministic offline responder
src/main.py	FastAPI application and endpoints
src/integration/	Context builder and mock gateway APIs
src/db/	SQLAlchemy models, repositories, seed data
frontend/src/	React console: App, hooks, API client, presets, components

6.2 Financial Calculators

The room-rent pro-rata rule (clause 6.2.4(d)) pays the associated medical expenses (room, nursing, doctors' fees and operation theatre) in the ratio of the eligible room rent to the rent actually charged:

Listing 6.1: Room-rent pro-rata (calculators.py, simplified)

```
associated = round(room + nursing + practitioner_fees + ot_charges, 4)
if actual_room_rent > eligible_room_rent:
    ratio = min(1.0, eligible_room_rent / actual_room_rent)
    payable = round(ratio * associated, 4)
    deduction = associated - payable
else:
    payable, deduction = associated, 0.0
```

Co-payments stack: the base co-pay, the room-category co-pay from Annexure V of the wording, the tiered-network penalty, the HeadsUp penalty and the late-intimation penalty add up, and the total is applied to the amount remaining after the deductible. Any Cash-Bag+ wallet balance is used to offset the co-pay. The SI waterfall then draws the payable amount from the base sum insured, the Booster+ pool and the ReAssure Forever pool in that order, recording how much came from each.

6.3 Planner

Listing 6.2: Deterministic topological sort with a priority queue (planner.py, simplified)

```
heap = [(priority(r), r) for r, deg in in_degree.items() if deg == 0]
heapq.heapify(heap)
order = []
while heap:
    _, rule = heapq.heappop(heap)          # lowest (gate, priority, id) first
    order.append(nodes[rule])
    for nxt in adjacency[rule]:
        in_degree[nxt] -= 1
        if in_degree[nxt] == 0:
            heapq.heappush(heap, (priority(nxt), nxt))
if len(order) != len(nodes):                # a dependency cycle
    order = sorted(nodes.values(), key=priority_tuple)   # fall back to priority order
```

6.4 Layer-Parallel Execution

Steps are grouped by depth in the DAG. All steps in a layer start together with `asyncio.gather`; each step reports its trace to the streaming callback the moment it finishes, so the console updates live. Results are then applied to the running admissible and payable amounts in plan order, because those amounts are stateful.

6.5 Semantic Agent and Structured Output

Listing 6.3: Groq call in JSON mode (semantic_agent.py)

```
response = self.groq_client.chat.completions.create(
    model=self.groq_model,                        # qwen/qwen3.8-27b by default
    messages=[{"role": "system", "content": f"{system_prompt}\n\nJSON Schema:\n{schema}"},
              {"role": "user", "content": user_prompt}],
    response_format={"type": "json_object"},
    temperature=0.0, seed=42, max_tokens=1024)
return self._extract_json_from_response(response.choices[0].message.content)
# caller: SemanticAdjudicationPayload.model_validate_json(raw) -> else manual review
```

6.6 Batched Exclusion Screening

Gate 5 has about eleven semantic exclusion rules per line item (maternity, cosmetic, rest cure, obesity, hazardous sports, breach of law, unproven treatment, and others). Calling the LLM once per rule re-sent the same claim facts eleven times. On Groq's free tier, which allows 8,000 tokens per minute, this meant a single claim could take minutes, and claims were sent to review because calls were rate-limited.

I changed the agent to **screen all semantic exclusion rules of a line item in one call**. The claim facts are sent once, followed by the policy text of each rule; the model returns one verdict per rule. The verdicts are stored in the agent's existing result cache under the same keys the per-rule steps use, so the rest of the pipeline did not change. If the batch call fails or omits a rule, those rules simply fall back to individual calls. This reduced LLM calls per

line item from about eleven to one; with a fresh rate budget a claim is adjudicated in roughly 1.5 to 3.5 seconds.

6.7 Streaming, Documents and Observability

The streaming endpoint runs the pipeline in a background task and pushes each trace into an asyncio queue that the SSE response drains. The document endpoint extracts text with pypdf, falls back to Tesseract OCR for scanned pages, truncates it to a safe length, and asks the LLM for a fixed set of fields. The reasoning logger writes one structured record per LLM call, and the telemetry engine writes one record per claim with per-gate and per-tool latency and token usage.

6.8 Offline Mode

For demonstrations without internet access, `LLM_PROVIDER=mock` uses a deterministic responder. It reads only the claim-details section of each prompt and only applies the keyword family that matches the rule being asked about. An earlier version matched words anywhere in the prompt and was misled by the rule text itself (the maternity rule's own exception mentions ectopic pregnancy), which made it unsuitable for demos.

Chapter 7: Testing and Results

7.1 Test Strategy

Table 7.1: *Levels of testing*

Level	What is checked	Tools
Unit	Each calculator and gate on hand-worked examples	pytest
Property-based	Invariants over thousands of random inputs	Hypothesis
Scenario	Complete claims through the pipeline against expected outcomes	pytest with the offline LLM
Integration / API	Endpoints, SSE stream, summary and extraction contracts	FastAPI TestClient
Provider	Groq request shape, missing-key handling, batching and fallback	pytest with mocks
End to end	Ten presets through the real UI and the live LLM	Playwright, Groq

Table 7.2: *Automated test suite*

Test file	Tests
test_calculators.py	14
test_hardening.py	11
test_integration_plumbing.py	2
test_metrics.py	4
test_new_features.py	15
test_phase2_integration.py	14
test_property_invariants.py	4
test_rule_scenarios.py	13
Total	77

All 77 tests pass. The suite runs in about 30 seconds without network access, because a shared fixture replaces the LLM with the deterministic responder.

7.2 Property-Based Tests of the Money

Example-based tests only check the cases the author thought of. For the calculators I wrote properties that must hold for any input, and let Hypothesis search for counter-examples:

- **Room pro-rata:** the ratio is always between 0 and 1; there is no deduction when the room rent is within the limit and always a deduction when it is breached; payable plus deduction equals the associated expenses to four decimal places.
- **Co-payment:** the co-pay never exceeds the admissible amount, and payable plus co-pay equals the admissible amount.

- **SI waterfall:** amount paid plus shortfall equals the payable amount, and no pool (base, Booster+, Forever) ever goes negative.
- **Deductible:** the deductible applied never exceeds the remaining annual limit or the claim, and money is conserved.

7.3 Scenario Results on the Live LLM

Table 7.3 shows the ten demonstration scenarios run through the console with the Groq provider (Qwen3.8 27B). Every scenario completed, and the decisions matched those from the deterministic offline mode, which shows that the LLM did not change any outcome where the facts were clear.

Table 7.3: *End-to-end results with the live LLM*

#	Scenario	Decision	Claimed	Payable
1	Room-rent pro-rata breach	PARTIALLY_APPROVED	₹54,000	₹2,700
2	Lapsed policy	REJECTED	₹80,000	₹0
3	Post-hospitalisation, missing dates	ASSISTED_REVIEW	₹8,000	₹0
4	ReAssure Forever waterfall	APPROVED	₹12,00,000	₹12,00,000
5	Clean approval (cholecystectomy)	APPROVED	₹1,00,000	₹1,00,000
6	Multi-line (surgery, physio, cash)	PARTIALLY_APPROVED	₹1,72,500	₹1,54,500
7	Deductible applied	PARTIALLY_APPROVED	₹50,000	₹40,000
8	Suite upgrade and co-payment	PARTIALLY_APPROVED	₹2,50,000	₹41,670
9	Porting credit waiver	APPROVED	₹60,000	₹60,000
10	Tiered network penalty	PARTIALLY_APPROVED	₹1,20,000	₹96,000

Some results are worth explaining. In scenario 1 a Classic member took a suite instead of the eligible single private room, so 75% of the associated expenses were deducted pro rata and a stacked co-payment of 80% (10% base, 50% room-category co-pay from Annexure V and a 20% tiered-network penalty) was applied to the rest. In scenario 3 the claim is for post-hospitalisation physiotherapy but the related admission and discharge dates are missing, so the engine cannot check the 180-day window and refers the claim for assisted review. In scenario 6 one of three line items (a rest-cure stay) is excluded, so the claim is partially approved.

7.4 Screenshots

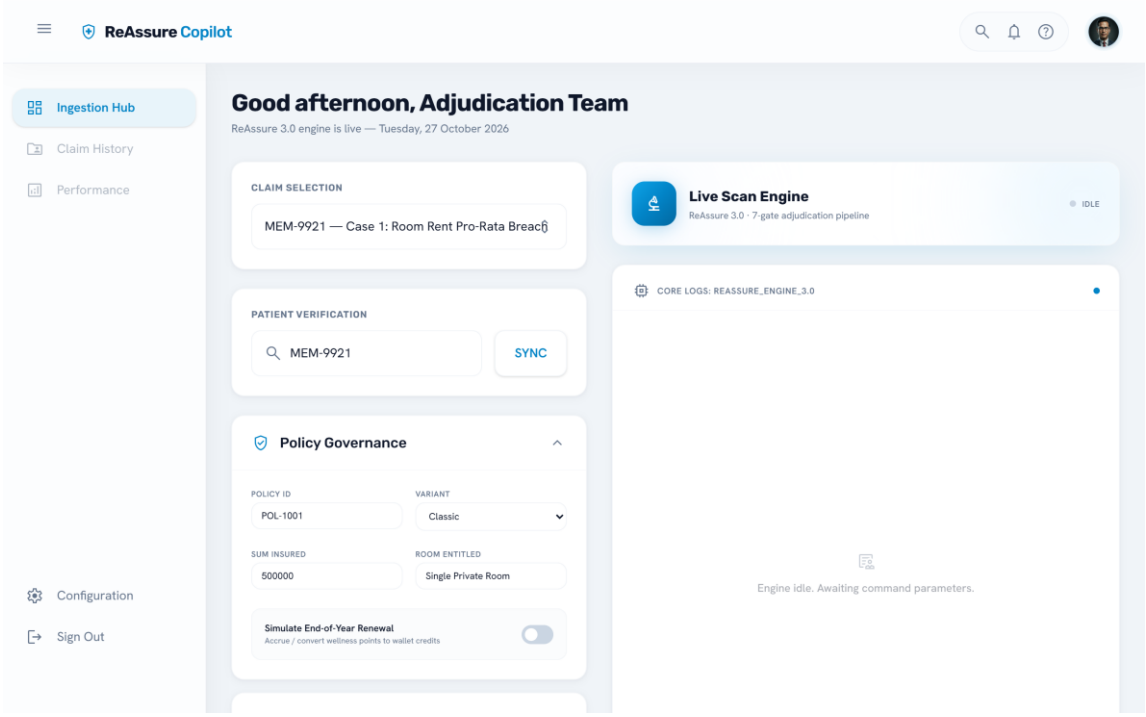


Figure 7.1: *Adjudication console: claim entry (left) and Live Scan Engine (right)*

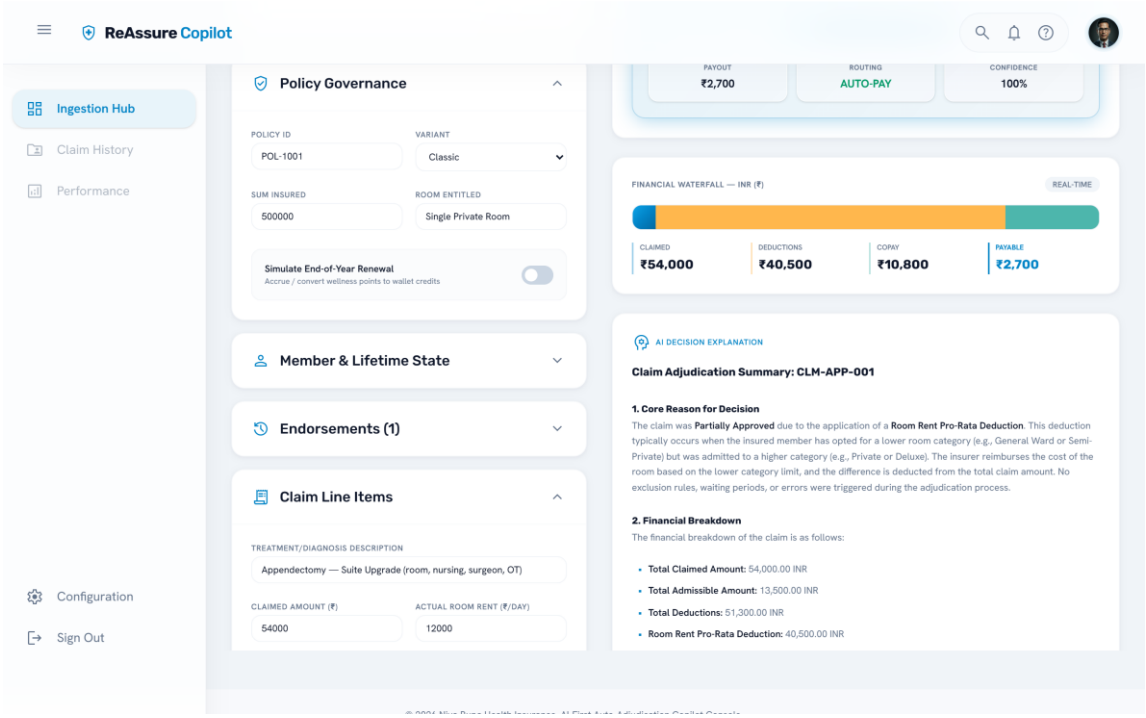


Figure 7.2: *Scenario 1: partial approval with financial waterfall and LLM-written explanation*

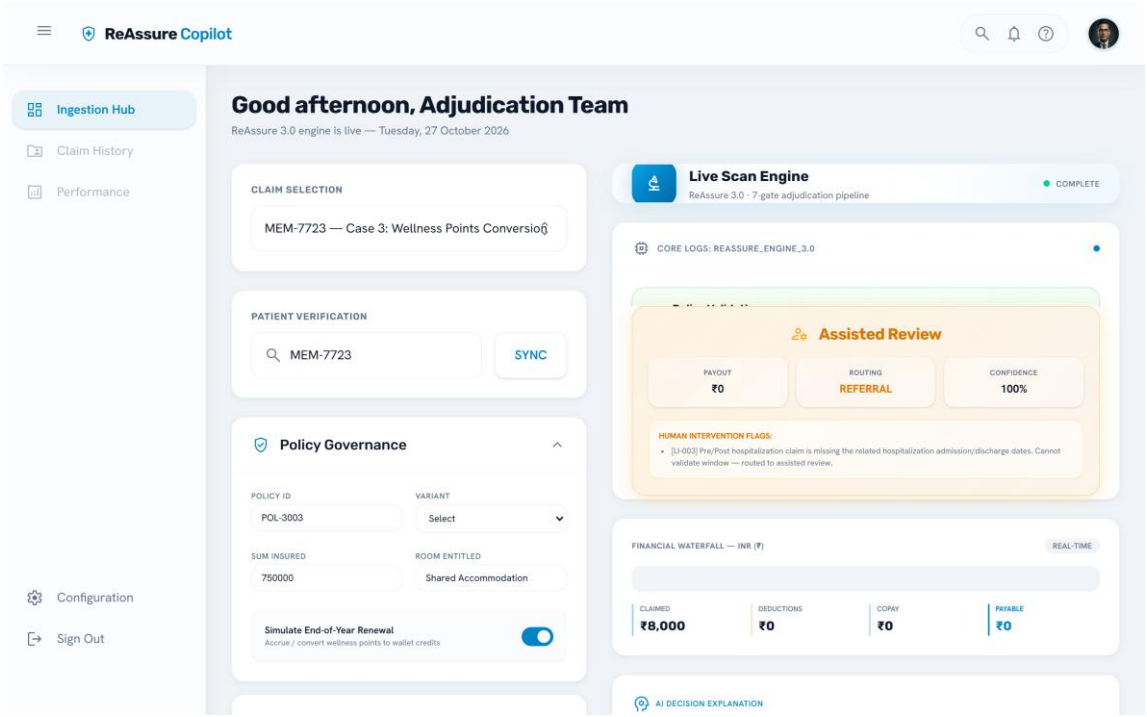


Figure 7.3: *Scenario 3: missing hospitalisation dates routed to assisted review*

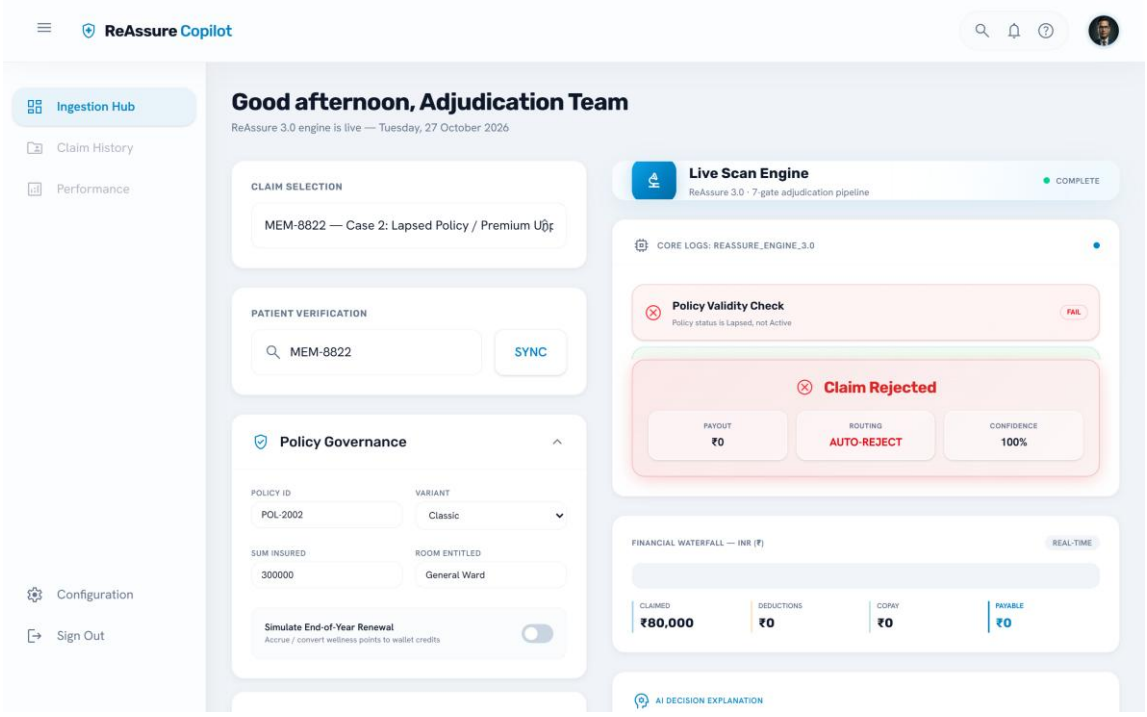


Figure 7.4: *Scenario 2: lapsed policy rejected at Gate 1 before any other rule runs*

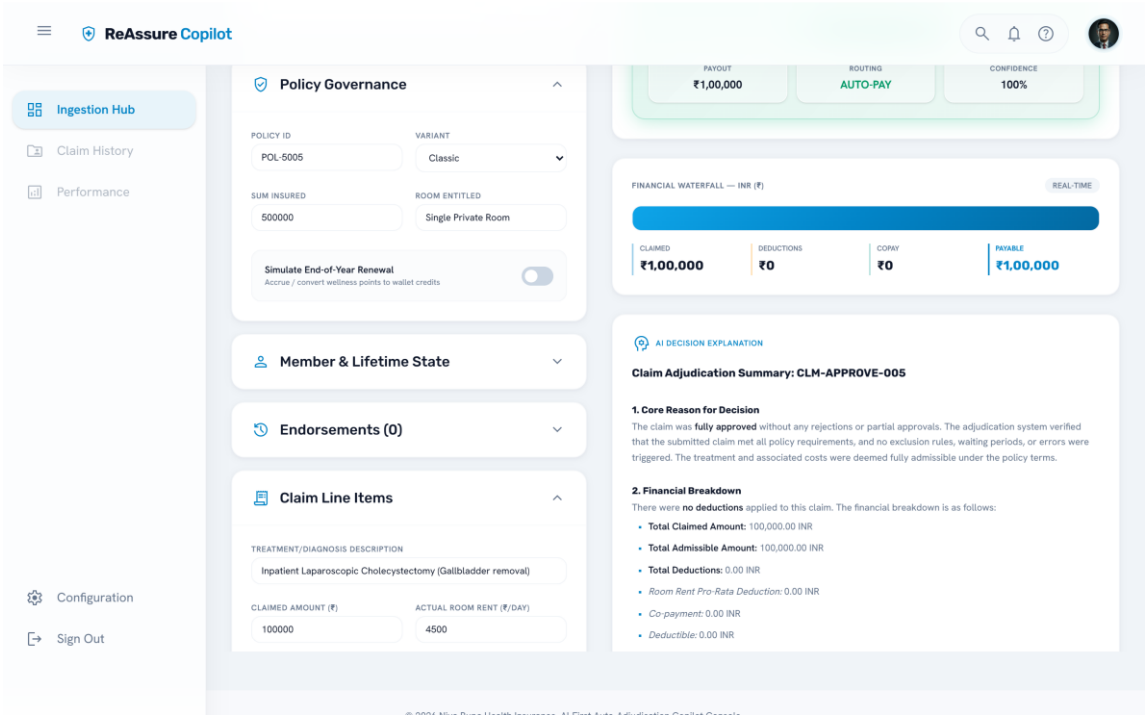


Figure 7.5: *Scenario 5: clean approval*

7.5 Defects Found and Fixed in Final Testing

Running the ten scenarios end to end and reading the traces exposed defects that the unit tests had not caught. Each fix came with a regression test that fails on the old code.

Table 7.4: *Defects found during final testing*

Defect	Effect	Fix
Pro-rata deduction not capped	Itemised expenses larger than the claimed amount gave a deduction above the claim, negative admissible amount and a wrong rejection	Deduction capped at the remaining admissible amount
Review request lost in Gate 6	Missing hospitalisation dates produced a ₹0 auto-rejection instead of review	Gate 6 review traces now route the line item to assisted review
Claim-level review flag	Claim marked as not needing review although a line item did; console showed AUTO-PAY	Flag also set when any line item needs review
Repeated trace entries	Gate-level checks repeated once per rule, about 17 identical rows in the live view	Each pass attributed to its own rule; optional-cover notes recorded once
Cache key used a prompt prefix	Two claims with long rule text could share a cached LLM verdict	Cache key hashes the full prompt
One LLM call per exclusion rule	Rate limits on the free tier; claims sent to review	Batched screening with per-rule fallback (Section 6.6)
Offline mode misread rule text	Absurd reasoning such as an ectopic-pregnancy verdict for appendicitis	Mock reads only claim details and the matching rule family
UI labels	Rejected claims shown as AUTO-PAY; deductions labelled as exclusions; bold markdown shown as bullets	Labels and markdown rendering corrected

Defect	Effect	Fix
Flaky property test	Occasional timeout under full-suite load	Hypothesis deadline disabled for pure calculators

7.6 Performance

Deterministic adjudication of a claim (offline mode) takes a few tens of milliseconds. With the live LLM, a claim takes about 1.5 to 3.5 seconds when the per-minute token budget is fresh, dominated by one batched exclusion call and, for non-approved claims, the explanation call. When many claims are sent back to back on the free tier, the client waits for Groq's rate-limit window and latency rises to tens of seconds; a paid tier or the local model removes this limit.

Chapter 8: Challenges and Learnings

8.1 Challenges

- **Turning prose into rules.** Policy clauses have exceptions and cross-references. A single clause such as the room-rent rule touches several bill components and interacts with the room-category co-pay in an annexure. Each rule needed careful reading and, often, a question to the mentor.
- **Making the LLM dependable.** Early local-model replies wrapped JSON in reasoning text or markdown. I solved this with a strict output contract, response cleaning, grammar-constrained decoding on llama.cpp, schema validation, and routing to review on any failure.
- **Stateful features.** ReAssure Forever, Lock the Clock and Booster+ depend on earlier claims, so state had to be modelled explicitly and persisted with row locks.
- **Concurrency.** Running rules in parallel exposed shared-state bugs; per-call state was moved into thread-local and per-claim objects.
- **Rate limits.** The free cloud tier allows 8,000 tokens per minute, which shaped the batching design.

8.2 Learnings

- Deciding what *not* to give an AI model is as important as the prompt. Keeping arithmetic deterministic made the system both more accurate and easier to explain.
- End-to-end scenarios and reading traces find defects that unit tests miss; most defects in Table 7.5 were found this way.
- Property-based testing is a cheap way to gain confidence in financial code.
- Good observability (structured logs, telemetry, decision graphs) pays for itself the first time something goes wrong.
- Professional practice: working from a specification, keeping documentation current, using version control properly and never committing secrets.

Chapter 9: Conclusion and Future Scope

9.1 Conclusion

The project delivered a working claims auto-adjudication engine for ReAssure 3.0 that encodes 92 product rules, computes payable amounts deterministically, uses an LLM only for clinical language with validated output, and routes uncertain cases to people. It runs on a cloud LLM, a local model or fully offline, streams its reasoning live to a web console, and records a complete audit trail. 77 automated tests pass and ten reference scenarios produce the expected decisions on the live LLM. The training gave me hands-on experience of building a reliable system around an AI model in a regulated domain.

9.2 Future Scope

- Connect the context builder to the real policy, member and balance systems instead of mock gateways.
- Complete the reconciliation with the Policy Administration System, currently a stub.
- Build a labelled evaluation set from historical claims to measure agreement with human processors and calibrate confidence thresholds.
- Add a reviewer workbench so human decisions on referred claims are captured and fed back.
- Run the local model on a GPU server in the organisation's own infrastructure to keep claim data in house.
- Support further products by adding new versioned rule files.

References

1. Niva Bupa Health Insurance, *ReAssure 3.0 Policy Wordings*, UIN NBHHLIP26047V012526.
2. Claims 2.0 AI-First Auto-Adjudication: Technical Solution Design, Product / Policy / Member Validation Layer, v1.0, May 2026 (internal).
3. Insurance Regulatory and Development Authority of India, *Master Circular on Health Insurance Business*, 2024.
4. A. B. Kahn, "Topological sorting of large networks," *Communications of the ACM*, 5(11), 1962.
5. FastAPI documentation, https://fastapi.tiangolo.com
6. Pydantic documentation, https://docs.pydantic.dev
7. Hypothesis documentation, https://hypothesis.readthedocs.io
8. Groq API documentation, https://console.groq.com/docs
9. llama.cpp project, https://github.com/ggml-org/llama.cpp
10. React documentation, https://react.dev

Appendix A: Running the Project

Backend

```
pip install -r requirements.txt
copy .env.example .env          # set GROQ_API_KEY, or LLM_PROVIDER=mock for offline
python -m uvicorn src.main:app --host 127.0.0.1 --port 8000
```

Frontend

```
cd frontend
pnpm install
pnpm run dev                    # http://localhost:5173
```

Tests

```
.venv\Scripts\python.exe -m pytest -q
```

Main configuration settings (.env)

Setting	Default	Meaning
LLM_PROVIDER	groq	groq, local or mock
GROQ_API_KEY / GROQ_MODEL	— / qwen/qwen3.8-27b	Cloud LLM credentials and model
LLM_URL	http://127.0.0.1:8080	llama.cpp server for local provider
AUTO_APPROVE_THRESHOLD	0.90	Minimum confidence to auto-decide
ASSISTED_REVIEW_THRESHOLD	0.70	Below this, medical review
MEDICAL_REVIEW_THRESHOLD	0.50	Below this, pending senior review
ADJUDICATION_API_KEY	(unset)	Enables API-key authentication